AWS SAA（SAA-C03）　分野2 弾力性に優れたアーキテクチャの設計　#10

高可用性と災害対策

部品が壊れても止まらない（高可用性）と、大きな災害のあとに戻す（ディザスタリカバリ）。RTO・RPO と 4 つの DR 戦略、バックアップとレプリケーションの使い分け

各ページの下に、根拠にした AWS の公式ドキュメント（ホワイトペーパー「AWS でのワークロードのディザスタリカバリ」・AWS Backup・Well-Architected 信頼性の柱など）を書いた（青い文字を押すと開く）。「」の中はドキュメントの原文どおり（日本語版は機械翻訳のため、語順が不自然なところもそのまま）。会社名や構成の例は Claude が作った例

全体像：何が壊れるかで対策が変わる 試験ガイド タスク 2.2

部品（1 台・1 ディスク）	AZ（データセンター）	リージョン全体	データ（削除・破損）
複数台＋ALB	マルチ AZ に分ける	別リージョンへの DR	ポイントインタイム
Auto Scaling で置き換え	静的安定性	4 つの戦略から選ぶ	Vault Lock・別アカウント
#09 で学んだ	高可用性（第1章）	DR（第2章）	バックアップ（第3章）

高可用性と DR は別もの：高可用性はワークロードの部品に、DR はワークロード全体の別のコピーに重点を置く。まず高可用性の目標を満たし、そのうえで DR を考える。

レプリケーションはバックアップではない：継続的なレプリケーションは、削除や破損もそのまま相手にコピーする。DR にはポイントインタイムのバックアップも要る。

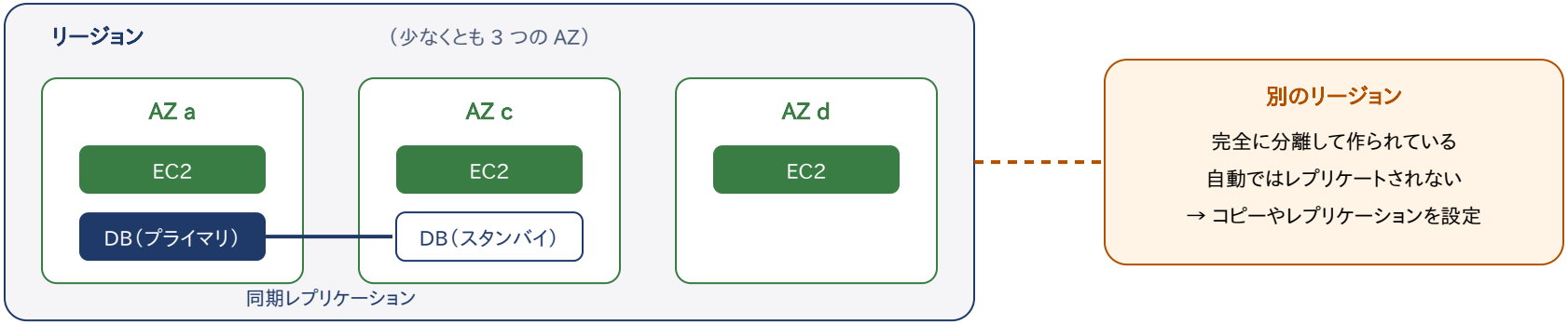
出典：高可用性はディザスタリカバリではありません（「プライマリストレージデバイスでファイルを削除または破損した場合、それらの破壊的な変更はセカンダリストレージデバイスにレプリケートできます。」「代わりに、DR 戦略の一部としてpoint-in-timeバックアップも必要です。」）

第1章

高可用性の基本

リージョンと AZ のしくみ、単一障害点をなくす、障害時に「起動に頼らない」設計。DR を支える自動化と見える化の道具。

リージョンと AZ



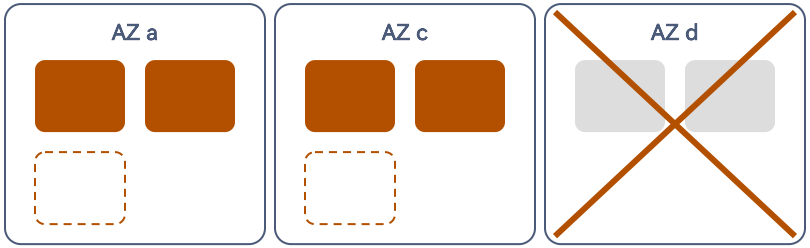
AZ の距離：相関する障害を防ぐため最大でおよそ 100 km 離れているが、1 桁ミリ秒のレイテンシーで同期レプリケーションできるほど近い。だからマルチ AZ の DB は同期でコピーできる。

単一障害点（SPOF）：1 つの AZ の 1 台の EC2・1 つの NAT ゲートウェイ・シングル AZ の DB。1 つの AZ だけのワークロードには高可用性がない。データセンター 1 つの障害までなら、マルチ AZ で守れる。

出典：AWS アベイラビリティーゾーン（「各リージョンには少なくとも 3 つのアベイラビリティーゾーンがあります。」）／アベイラビリティーゾーン（「リージョン内のアベイラビリティーゾーン間は、相関する障害を防ぐために意味のある距離として最大 60 マイル (約 100 km) 離れていますが、1 桁ミリ秒のレイテンシーで同期レプリケーションを利用するには十分近い距離にあります。」）／リージョンとゾーン（「各リージョンは他のリージョンと完全に分離されるように設計されています。」）

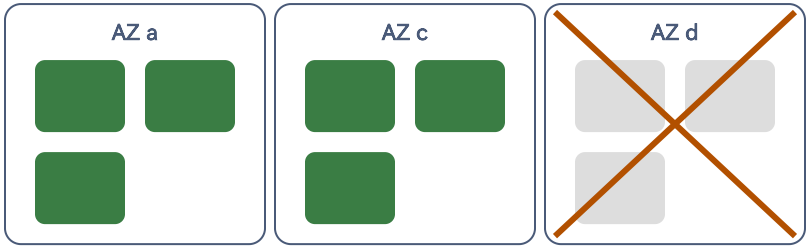
静的安定性：障害時の「起動」に頼らない

起動に頼る（安いが確実ではない）



6 台中 2 台が消え、残り 4 台。足りない分を起動する
（起動の仕組みと残りの AZ のキャパシティに頼る）

静的安定性（前もって多めに置く）



9 台中 3 台が消えても、残り 6 台で全負荷を受ける
（何も起動しなくてよい）

常に 6 台が必要なワークロードの例（Claude が作った例）。

なぜ：新しいインスタンスの起動に頼る設計は安いが、AZ やリージョンの大規模な障害では、起動の仕組みと、影響のないゾーンのキャパシティの両方に頼るので効果が下がる。

コストとの交換：AZ を増やすほど、静的安定性のために余分に置く台数は減る（Claude の計算：2 AZ なら必要数の 2 倍、3 AZ なら 1.5 倍）。

出典：REL11-BP05 静的安定性を使用してバイモーダル動作を防止する（「バイモーダル動作の例は、安定した Amazon EC2 設計により、1 つの AZ が削除された場合にワークロードの負荷を処理するのに十分な数のインスタンスが各 AZ にプロビジョニングされる場合です。」「ゾーンを追加することで、静的安定性に必要な追加のコンピューティング量を減らすことができます。」）

DR を支える自動化：作り直せるようにしておく

道具	何をする	本番の言い回し
CloudFormation（IaC）	インフラをテンプレートで書き、同じ環境を確実に作り直す。CDK はプログラミング言語で書き、CloudFormation に変換	「DR リージョンに同じ環境をすばやく」
StackSets	複数のアカウントとリージョンに、1 回の操作でスタックを作る・更新する・消す	「複数アカウント・複数リージョンに同じ構成」
AMI のコピー	AMI はリージョン固有。別のリージョンで起動するにはコピーしておく	「大阪でも同じ AMI から起動」
Service Quotas	リージョンごとのクォータ（制限）を見て、引き上げを申請する	「DR リージョンで本番の規模まで増やせない」

コントロールプレーンとデータプレーン：作る・変える（API での起動や設定）はコントロールプレーン、動いているものがサービスを提供するのはデータプレーン。データプレーンのほうがふつうは可用性の設計目標が高いので、フェイルオーバーはデータプレーンの操作だけで済むようにする。

ひっかけ：「災害時に Auto Scaling で増やす」はコントロールプレーンに頼る。確実さを優先するなら前もって置く（静的安定性・ホットスタンバイ）。

出典：StackSets を使用したアカウントとリージョン全体でのスタックの管理／Amazon EC2 AMI のコピー（「ただし、AMI はリージョンに固有のリソースであるため、特定の AWS リージョンでインスタンスを起動するときは、そのリージョンに AMI を配置する必要があります。」）／Service Quotas とは何ですか。／クラウド内での災害対策オプション（「これは、データプレーンには通常、コントロールプレーンよりも高い可用性の設計目標があるためです。」）

見える化・接続・確かめる道具

サービス	何をする	本番の言い回し → これ
Amazon RDS Proxy	DB への接続をプールして共有。フェイルオーバーのとき、アプリの接続を保ったままスタンバイにつなぐ。IAM 認証・Secrets Manager	「大量の Lambda で接続があふれる」「フェイルオーバーの切断を減らす」
AWS X-Ray	リクエストのデータ（トレース）を集め、サービスをまたぐ経路のどこが遅いか・エラーかを見せる	「どのサービスで時間がかかっているか」
AWS FIS	わざと障害を起こす実験。条件を満たしたら自動で止めるガードレール	「本当に AZ の障害に耐えるか実際に試す」
AWS Resilience Hub	RTO・RPO の目標を決め、満たせるかを評価し、Well-Architected に基づいて改善を勧める	「目標を満たせるか評価・追跡」

まぎらわしい組：CloudWatch＝メトリクスとログ。X-Ray＝リクエストの経路。CloudTrail＝だれがどの API を呼んだか。Config＝設定の変化。

RDS Proxy の位置：読み取りを分けたいならリードレプリカ、接続の数とフェイルオーバーの切断なら RDS Proxy。インスタンスを大きくするのは一時しのぎ。

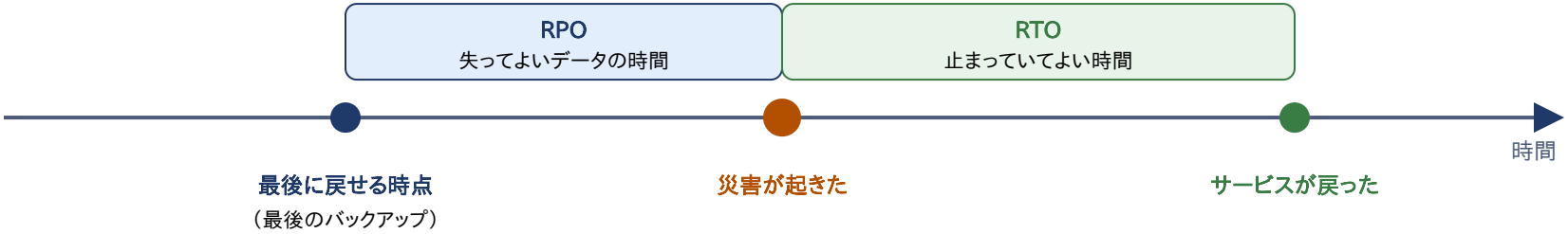
出典：Amazon RDS Proxy（「RDS Proxy は、アプリケーション接続を維持しながらスタンバイ DB インスタンスに自動的に接続することで、データベースの障害に対するアプリケーションの耐障害性を高めます。」）／AWS X-Ray とは／AWS Fault Injection Service とは／AWS Resilience Hub とは（「AWS Resilience Hub を使用すると、レジリエンス目標を定義し、その目標に対するレジリエンス体制を評価し、AWS Well-Architected フレームワークに基づいて改善のための推奨事項を実装できます。」）

第2章

DR の目標と 4 つの戦略

どれだけ早く戻すか（RTO）、どこまでのデータを失ってよいか（RPO）。
目標とコストで、4 つの戦略から選ぶ。

RPO と RTO



	意味	小さくするには
RPO	最後のデータの復旧時点から、障害までに許される最大の時間（どこまで戻るか）	バックアップの頻度を上げる → 継続的なレプリケーション
RTO	サービスが止まってから復旧までに許される最大の時間（どれだけ早く戻すか）	DR リージョンに前もって動かしておくものを増やす

コストで決める：RTO・RPO は事業継続計画（BCP）のビジネスへの影響分析で決める。復旧の仕組みのコストが障害や損失のコストより高いなら、規制などがない限り作らない。

出典：事業継続計画 (BCP)（「目標復旧時間 (RTO) は、サービスの中断からサービスの復旧までの最大許容遅延時間です。」「目標復旧時点 (RPO) は、最後のデータ復旧時点からの最大許容時間です。」「復旧戦略のコストが障害や損失のコストよりも高い場合は、規制要件などのセカンダリドライバーがない限り、復旧オプションを設定しないでください。」）

4 つの DR 戦略

バックアップと復元

DR 側：バックアップだけ

災害時：インフラを作り
データを復元

パイロットライト

DR 側：データは常に複製

サーバーは「スイッチオフ」
災害時：オンにして増やす

ウォームスタンバイ

DR 側：縮小版の本番が
常に動く（すぐ処理できる）
災害時：スケールアップ

マルチサイト A/A

全リージョンで本番の
トラフィックを処理
災害時：壊れた所を外す



コスト・複雑さ 低い／復旧 遅い

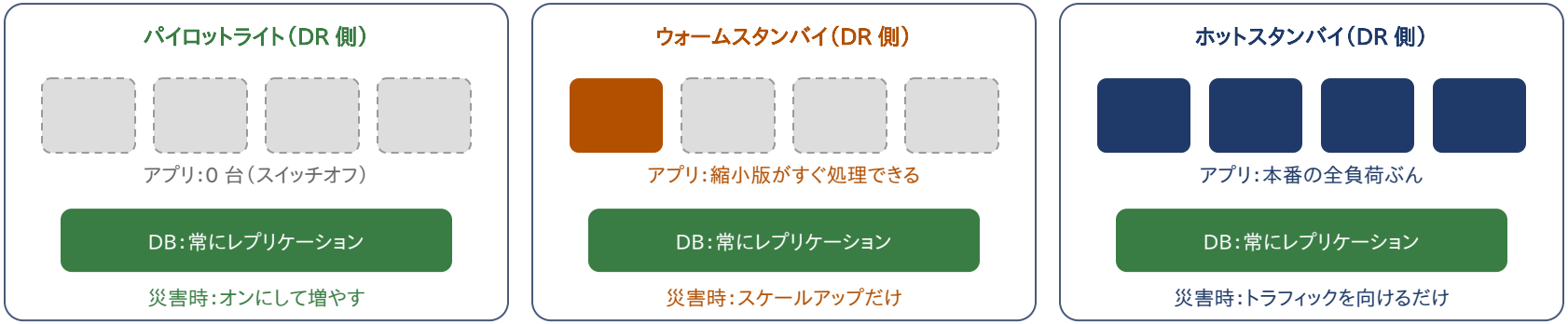
コスト・複雑さ 高い／復旧 ほぼゼロ

選び方：要件の RTO・RPO を満たす中で、最も安いもの。データセンター 1 つの障害だけなら、マルチ AZ ＋バックアップと復元で足りることもある。リージョンの災害や規制が要件なら、パイロットライト以上。

どの戦略でもバックアップ：マルチサイトでも、データの破損や削除からの復旧時間は 0 より大きく、戻せるのは災害が見つかる前の時点。バックアップと復旧のテストは要る。

出典：クラウド内での災害対策オプション（「AWS 内で利用できるディザスタリカバリ戦略は、低コストと複雑さの低いバックアップから、複数のアクティブなリージョンを使用したより複雑な戦略まで、4 つのアプローチに広く分類できます。」「このアプローチはディザスタリカバリに対する最も複雑でコストのかかるアプローチですが、正しいテクノロジーの選択と実装により、ほとんどのディザスタの復旧時間をほぼゼロに短縮できます (ただし、データの破損はバックアップに依存する必要があり、通常はゼロ以外の復旧ポイントになる可能性があります)。」）

パイロットライト・ウォームスタンバイ・ホットスタンバイ



パイロットライト vs ウォームスタンバイ：どちらも DR リージョンにプライマリのコピーを持つ。違いは、パイロットライトは追加の操作なしではリクエストを処理できず、ウォームスタンバイは（小さい規模で）すぐに処理できること。

ホットスタンバイ：Auto Scaling はコントロールプレーンなので、それに頼ると回復性が下がる。全負荷ぶんを前もって置く静的に安定した形がホットスタンバイ。ふだんは 1 つのリージョンだけが処理する。

出典：クラウド内での災害対策オプション（「違いは、パイロットライトは追加のアクションを最初に実行しない限りリクエストを処理できないのに対し、ウォームスタンバイはトラフィックを (キャパシティーレベルを減らして) すぐに処理できることです。」「Auto Scaling はコントロールプレーンのアクティビティであるため、それに依存すると、全体的な復旧戦略の耐障害性が低下します。」「この静的に安定した設定は、ホットスタンバイと呼ばれます (次のセクションを参照)。」）

マルチサイトアクティブ/アクティブとデータの書き方

書き込みの戦略	どこに書く	向くサービス
書き込みグローバル	すべての書き込みを 1 つのリージョンへ。そこが落ちたら別のリージョンを昇格	Aurora グローバルデータベース（1 分以内に昇格、書き込み転送）
書き込みローカル	利用者に最も近いリージョンへ書く	DynamoDB グローバルテーブル（同時更新は最後に書いたものが勝つ）
書き込みパーティション	ユーザー ID などのキーで、書くリージョンを決めて競合を避ける	双方向の S3 レプリケーション

フェイルオーバーはない：全リージョンでトラフィックを処理しているので、壊れたリージョンからトラフィックを外すだけ。テストでは「外せるか」「残りで全部を受けられるか」を確かめる。

アクティブ/パッシブとの違い：アクティブ/パッシブ（パイロットライト・ウォーム・ホット）では、ふだんは全トラフィックがプライマリへ行き、書き込みもプライマリだけ。

出典：クラウド内での災害対策オプション（「マルチサイトがアクティブ/アクティブの場合、ワークロードが複数のリージョンで実行されているため、このシナリオではフェイルオーバーのようなものはありません。」「Amazon DynamoDB グローバルテーブルでは、最後のライターが同時更新間の調整を獲得します。」「アクティブ/パッシブ戦略では、書き込みはプライマリリージョンに対してのみ行われます。」）

フェイルオーバーと DR のテスト

フェイルオーバーの注意

自動は慎重に: ヘルスチェックやアラームで自動に始めるフェイルオーバーは慎重に使う。復旧時間と復旧ポイントは 0 より大きいので、誤ったアラームで切り替えると、その分の損失が出る。手動で始めることも多い。

フェイルバック: 復旧したら、プライマリに戻す手順も用意し、試しておく。

テストしないと気付けないこと（例）

症状	対策
DR リージョンで台数が増やせない	サービスクォータを前もって引き上げる
DR 側の AMI が古い	AMI を作るたびにコピー
前は足りたキャパシティが今は足りない	本番の成長に合わせて見直す
バックアップから戻せない	復元のテストを定期的に

テストの考え方: くり返しテストしている経路しか、いざというときにうまく動かない。DR リージョンへのフェイルオーバーを定期的に行い、RTO と RPO を満たせるか確かめる。

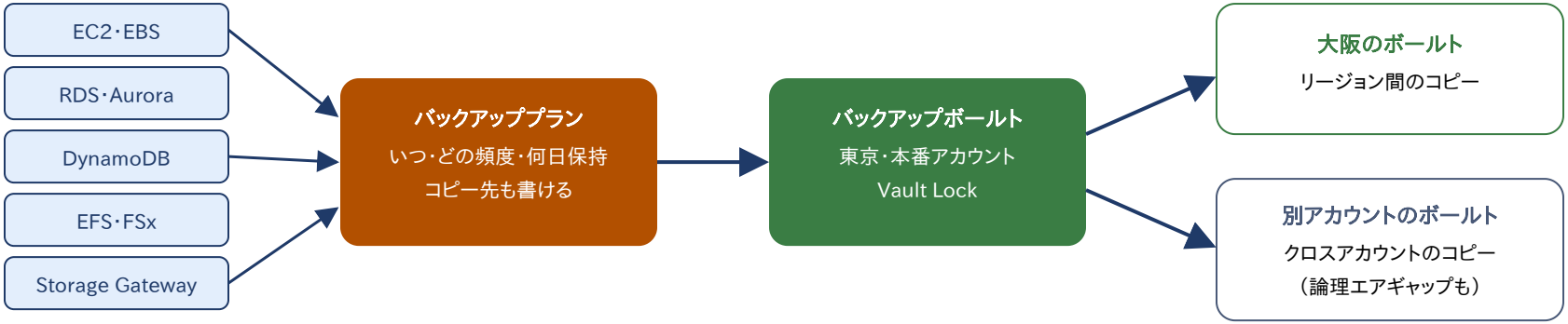
出典: クラウド内での災害対策オプション（「ヘルスチェックまたはアラームに基づいて自動的に開始されるフェイルオーバーは、慎重に使用する必要があります。」）／ディザスタリカバリのテスト（「エラー復旧がうまくいくのは頻繁にテストする経路のみであることは、これまでの経験からも明らかです。」「最後にテストしたときに十分だったセカンダリの容量が、このシナリオでは負荷を許容できなくなるか、セカンダリリージョンのサービスクォータが十分ではない可能性があります。」）

第3章

データの保護と複製・切り替え

バックアップを一元化して消されないように守る。データを別のリージョンに複製し、利用者のトラフィックを切り替える。

AWS Backup：まとめて守る



一元化：サービスごとにバラバラにスナップショットを取らず、バックアッププランで自動でバックアップ・保持する。タグでリソースを選べる。バックアップの頻度が RPO を決める。

コピーの目的：リージョン間＝本番から距離を離す事業継続・規制の要件。アカウント間＝本番アカウントが侵害されても消されないように。

出典：AWS Backup とは（「AWS Backup は、サービス間、クラウド内、オンプレミスでデータ保護を簡単に一元化および自動化できるフルマネージド AWS サービスです。」）／AWS リージョン間でのバックアップコピーの作成（「スケジュールバックアッププランを使用して、バックアップを AWS リージョン間でコピーできます。」）／AWS アカウント間でのバックアップコピーの作成（「運用上またはセキュリティ上の理由から、組織 AWS アカウント 内の 1 つ以上の にバックアップを安全にコピーする場合は、クロスアカウントバックアップを使用します。」）

消されないバックアップ：Vault Lock と論理エアギャップ

しくみ	だれが消せる・変えられる	本番の言い回し
ふつうのボールト	十分な権限を持つユーザー（ルートユーザーなど）は削除できる	—
Vault Lock ガバナンスモード	十分な IAM 権限を持つユーザーはロックを解除できる	「権限のある管理者は外せてよい」「まず試す」
Vault Lock コンプライアンスモード	猶予期間が過ぎると、顧客・アカウントの所有者・AWS も、設定の変更や削除ができない	「ルートユーザーを含めだれも消せない」「規制で保持」
論理エアギャップボールト	コンプライアンスモードが備わる特別なボールト。他のアカウントと共有し、持ち主のアカウントに入れなくても（MPA で）復旧できる	「アカウントが乗っ取られても別のアカウントから復元」

継続的バックアップ（PITR）：最初にフルバックアップ、あとはトランザクションログ。最大 35 日の範囲で、1 秒単位で指定した時刻に戻せる。スナップショットと両方で守ることが勧められている。

注意：ロックしたボールトのバックアップは保持期間が終わるまで消せないので、保持の設定を誤るとストレージの料金がかかり続ける。

出典：AWS Backup ボールトロック（「コンプライアンスモードでロックが有効になっていて、猶予期間が終了すると、復旧ポイントが含まれている限り、顧客、アカウント/データ所有者、または AWS によるボールト設定の変更または削除ができなくなります。」「ロックされたボールトのバックアップはライフサイクルが完了するまで削除できないため、意図しないストレージコストを避けるために、保持設定が適切に設定されていることを確認してください。」）／論理エアギャップボールト／継続的バックアップとポイントインタイムリカバリ (PITR)

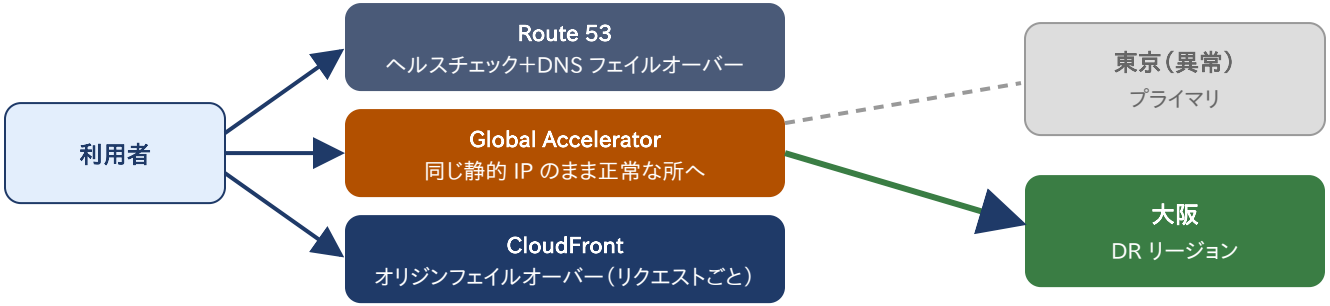
クロスリージョンのレプリケーション

サービス	しくみ	切り替え
S3 のレプリケーション（CRR）	DR リージョンのバケットへ、継続的に非同期でコピー。バージョニングで、誤った削除や変更からも守る	DR 側のバケットを使う（#13）
RDS のクロスリージョンリードレプリカ	別のリージョンに読み取り用のレプリカ	プライマリに昇格（数分・再起動あり）（#15）
Aurora グローバルデータベース	ふつう 1 秒未満で、最大 5 つのセカンダリリージョンへ	1 分以内に読み書きを引き受ける（#15）
DynamoDB グローバルテーブル	全リージョンで読み書きできる	切り替え不要（#16）
AWS Elastic Disaster Recovery	サーバーのブロックレベルのレプリケーション。安いストレージと最小限のコンピューティングのステージングエリアに置く	災害時に起動（パイロットライト）。RDS は対象外

レプリケーションとバックアップは両方：レプリケーションは RPO をほぼゼロにするが、削除や破損もそのままコピーされる。ポイントインタイムのバックアップを組み合わせる。

出典：クラウド内での災害対策オプション（「Amazon Aurora グローバルデータベースでは、プライマリリージョンでパフォーマンスの低下や機能停止が発生した場合、いずれかのセカンダリリージョンを昇格させて、リージョンが完全に機能停止した場合でも 1 分以内に読み取り/書き込みの責任を引き受けることができます。」「Elastic Disaster Recovery は、パイロットライト戦略を使用して、ステージングエリアとして使用される Amazon Virtual Private Cloud (Amazon VPC) 内のデータのコピーと「スイッチオフ」リソースを維持します。」）

利用者のトラフィックを切り替える



使い分け：Route 53＝DNS の名前で切り替え（#11）。Global Accelerator＝エニーキャストの同じ静的な IP のまま切り替え、DNS のキャッシュの問題も避ける。トラフィックダイヤルで割合も変えられる（#19）。CloudFront＝リクエストが失敗したらセカンダリのオリジンへ（#19）。

CloudFront は「リクエストごと」：オリジンフェイルオーバーでは、後に続くリクエストもまずプライマリに送られる。リージョン全体を切り替える操作ではない。

出典：クラウド内での災害対策オプション（「AnyCast IP を使用すると、1 つ以上の AWS リージョンの複数のエンドポイントを同じ静的パブリック IP アドレスに関連付けることができます。」「Global Accelerator は、DNS システム (Route 53 など) で発生する可能性のあるキャッシュの問題も回避します。」「前述のフェイルオーバーオペレーションとは異なり、後続のすべてのリクエストは引き続きプライマリエンドポイントに送信され、フェイルオーバーはリクエストごとに行われます。」）

まとめ

高可用性
マルチ AZ で単一障害点をなくす。静的安定性で、障害時の起動に頼らない

目標
RPO＝どこまで戻るか（バックアップの頻度）、RTO＝どれだけ早く戻すか

4 つの戦略
バックアップと復元 → パイロットライト → ウォームスタンバイ → マルチサイト。安い順＝遅い順

データと切り替え
AWS Backup・Vault Lock・PITR、レプリケーション、Route 53・Global Accelerator

ひっかけ	正しくは
RPO＝止まっていてよい時間	**それは RTO**
レプリケーションがあればバックアップ不要	**破損もコピーされる**
パイロットライトはすぐ処理できる	**それはウォーム**
ガバナンスモードならだれも消せない	**コンプライアンスモード**
AMI は全リージョンで使える	**コピーが要る**
DR リージョンのクォータは自動で同じ	**前もって引き上げ**

> 次の #11 では、Route 53 を学ぶ。

出典：このPDFの各ページの資料